JWT 게시판 구현 보고서

오늘의 맛집 후기

1 서비스 소개와 구현 범위

학번: 202410034　이름: 승연　제출일: 2026-10-28

서비스 이름: 오늘의 맛집 후기

기술 조합과 버전: Vue 3.5.43, Vite 8.3.4 / Node.js Express 5.3.0 / Node.js v24.18.0, jsonwebtoken 9.0.3, sql.js 1.14.2, SQLite 파일 DB

GitHub 저장소 URL: https://github.com/nguyenthingocduyen0405/CRUD

소스 코드 기준: main 브랜치. 최종 제출 기준 커밋 SHA 는 LMS 에 제출한다.

이 서비스는 다녀온 맛집과 음식을 기록하고 공유하는 게시판이다. 대상 사용자는 학교 주변이나 여행지에서 먹을 곳을 찾는 학생들이다. 후기는 누구나 읽을 수 있고, 글 작성은 로그인 사용자만, 수정과 삭제는 작성자 본인만 가능하도록 서버에서 검사한다. 수업에서 실습해 익숙한 Vue 와 Express 를 선택했고, 제공 템플릿의 CRUD 와 저장소 구조는 유지한 채 인증 TODO 와 안내 문구만 변경했다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	backend/src/app.js	토큰 없이 200 조회
로그인과 JWT 발급	완료	backend/src/auth.js issueToken	HS256, TTL 90 초
인증과 내 정보 조회	완료	backend/src/auth.js currentUserId	요청마다 검증
본인 글 작성, 수정, 삭제	완료	backend/src/auth.js assertOwner	타인 글은 403
만료, 로그아웃, 오류 표시	완료	frontend/src/api.js, App.vue	401 시 상태 해제
DB 데이터 유지	완료	backend/src/store.js	data/board.sqlite

2 구조와 변경 내역

전체 구조는 브라우저(127.0.0.1:4173 빌드 화면), Vite preview 서버의 /api 프록시, Express API 서버(127.0.0.1:8000), sql.js 기반 SQLite 파일(backend/data/board.sqlite) 순서로 연결된다.

로그인 요청 흐름: 로그인 폼의 email 과 password 를 api.js 의 login 이 POST /api/auth/login 으로 보내고, 서버는 verifyCredentials 로 계정을 확인한 뒤 issueToken 이 JWT 를 발급한다. 프런트는 응답의 accessToken 과 user 를 자바스크립트 메모리 변수 session 에만 저장하므로(localStorage 미사용) 새로고침하면 로그인이 해제된다.

게시글 수정 요청 흐름: 수정 저장 시 request 가 PUT /api/posts/{id}를 보내며 authorizationHeaders 가 메모리의 토큰으로 Authorization: Bearer 헤더를 붙인다. 서버는 currentUserId 의 토큰 검증, 글 존재 확인, assertOwner 의 작성자 비교, 입력 검사를 모두 통과했을 때만 DB 를 변경한다. 토큰은 /api/me 와 글 등록, 수정, 삭제 요청에만 전송된다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend/src/auth.js issueToken	501 NOT_IMPLEMENTED 반환	HS256 JWT 발급, JWT_SECRET 형식 검사	BE-01
backend/src/auth.js currentUserId	데모 ID 1 을 고정 반환	서명, 만료, 클레임 검증 후 ID 반환	BE-02
backend/src/auth.js assertOwner	빈 함수	작성자 불일치 시 403 FORBIDDEN	BE-03
frontend/src/api.js login, authorizationHeaders, request, canEdit	미구현(오류, 빈 값, 항상 true 반환)	토큰 메모리 저장, Bearer 헤더, 401 세션 해제, 버튼 표시	FE-01~FE-04
frontend/src/App.vue	데모 배너, 단일 오류 문구	서비스 안내, code 별 안내, 404 시 목록 갱신	FE-03, FE-04

작성 출처 구분: 제공 코드(라우트, 저장소, 계정 검증, 화면 골격)는 유지하였으며, 인증 및 프런트엔드 기능 구현 과정에서 AI 가 제시한 예시 코드와 구현 방법을 참고하였다. 작성한 코드는 API 명세와 비교하고 테스트를 통해 동작 여부를 확인하였다. 보조 파일로 backend/.env(저장소 미포함)와 docs/evidence/api-test-log.txt 를 추가하였다.

3 핵심 코드 설명

이 장의 코드는 보고서 분량을 위해 핵심 부분만 발췌한 것이며, 오류 응답의 message 문자열은 "..."로 축약했다. 전체 코드는 저장소의 해당 파일에 있다.

3.1 로그인과 JWT 발급

파일 / 함수: backend/src/auth.js issueToken(호출: app.js 로그인 라우트)

```
export function issueToken(user) {
  return jwt.sign({}, secretKey, {
    algorithm: "HS256",
    subject: String(user.id),
    expiresIn: Number(process.env.TOKEN_TTL || 90),
    issuer: ISSUER,
    audience: AUDIENCE,
  });
}
```

입력: email 과 password 다. fields 가 두 필드만 허용하고 형식 오류는 400 BAD_INPUT 이다. 처리: verifyCredentials 가 비밀번호를 확인하고 실패하면 토큰 없이 401 INVALID_CREDENTIALS 다. 확인된 사용자에게만 HS256 과 서버 비밀 키로 sub, exp(90 초), iss, aud 를 담아 서명하며 비밀번호는 넣지 않는다. 출력: 200 과 accessToken, expiresIn, user 다. 비밀 키는 .env 의 JWT_SECRET 이며 64 자리 16 진수만 허용하고 예시 키는 서버 시작 시 거부한다.

3.2 브라우저 토큰 저장과 전송

파일 / 함수: frontend/src/api.js 의 login, authorizationHeaders, request, clearSession

```
let session = { accessToken: "", user: null };

export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}

function authorizationHeaders() {
  return session.accessToken
    ? { Authorization: `Bearer ${session.accessToken}` }
    : {};
}
```

입력: 로그인 폼의 email 과 password 다. 처리: login 이 성공 응답의 토큰과 user 를 메모리 변수 session 에만 저장하므로 새로고침 시 해제되는 것이 의도된 동작이다. authorizationHeaders 는 토큰이 있을 때만 Bearer 헤더를 만들고, 로그아웃은 clearSession 으로 메모리를 비운다. 실패 조건: 로그인 실패(401)면 저장하지 않고, request 가 401 을 받으면 세션을 지워 화면도 로그아웃된다.

3.3 서버 JWT 검증

파일 / 함수: backend/src/auth.js currentUserId

```
export function currentUserId(req) {
  const match = /^Bearer₩s+(₩S+)$/.exec(req.get("Authorization") || "");
  if (!match)
    throw new ApiError(401, "TOKEN_REQUIRED", "로그인 토큰이 필요합니다.");
  let payload;
  try {
    payload = jwt.verify(match[1], secretKey, {
      algorithms: ["HS256"], issuer: ISSUER, audience: AUDIENCE,
    });
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError)
      throw new ApiError(401, "TOKEN_EXPIRED", "...");
    throw new ApiError(401, "INVALID_TOKEN", "...");
  }
  if (typeof payload.exp !== "number" || typeof payload.sub !== "string" ||
      !/^[1-9]₩d*$/.test(payload.sub))
    throw new ApiError(401, "INVALID_TOKEN", "...");
  const userId = Number(payload.sub);
  if (!findPublic(userId))
    throw new ApiError(401, "INVALID_TOKEN", "...");
  return userId;
```

```
}
```

입력: Authorization 헤더다. 처리: 헤더가 없거나 Bearer 형식이 아니면 401 TOKEN_REQUIRED 다. jwt.verify 가 HS256 고정으로 서명, 만료, iss, aud 를 검증하며(서명을 확인하지 않는 decode 는 미사용), 만료는 401 TOKEN_EXPIRED, 변조는 401 INVALID_TOKEN 으로 구분된다. verify 가 보장하지 않는 exp 존재와 sub 형식, 계정 존재도 추가로 검사한다. 출력: 검증된 사용자 ID 이며, 401 응답에는 WWW-Authenticate: Bearer 헤더가 붙는다.

3.4 작성자 권한 검사

파일 / 함수: backend/src/auth.js assertOwner, backend/src/app.js 수정 라우트

```
export function assertOwner(userId, authorId) {
  if (userId !== authorId)
    throw new ApiError(403, "FORBIDDEN",
      "본인이 작성한 글만 수정하거나 삭제할 수 있습니다.");
}
```

비교 대상: 검증된 사용자 ID 와 글의 authorId 다. 등록 시 작성자는 서버가 검증된 ID 로 저장하고 본문의 추가 필드는 400 으로 거부해 위조를 막는다. 수정과 삭제 라우트(app.js)는 인증(401), 글 존재(404), 작성자 비교(403), 입력 검증(400)을 모두 통과해야 DB 를 바꾼다. 401 은 신원 미확인, 403 은 권한 없음이다. 버튼을 숨겨도 직접 API 호출은 서버가 거부함을 T07 과 T08 로 확인했다.

4 테스트 결과와 증거

실행 환경: Windows 11, Node.js v24.18.0, 브라우저 Chromium(Playwright). backend 에서 npm ci 와 npm start, frontend 에서 npm ci 와 npm run build 와 npm run preview 로 실행했고, 빌드 성공 후 http://127.0.0.1:4173 접속과 /api 프록시 요청이 모두 200 으로 연결됨을 확인했다.

테스트 일시 및 코드 버전: API 검사 2026-10-24, 화면 검사 2026-10-25(UTC), 제출 코드와 동일 버전. 토큰 TTL 은 일반 90 초, 만료 테스트(T10)만 TOKEN_TTL=3 으로 재시작해 수행했다.

표의 실제 결과는 Python 스크립트로 직접 관찰한 HTTP 상태와 JSON code 이며 전체 기록은 docs/evidence/api-test-log.txt 에 있다. 화면 항목은 4173 번 빌드 화면을 Playwright 로 조작해 확인했고, 캡처(01~07 png)와 토큰을 가린 요청 기록(network-capture.txt)을 docs/evidence 에 포함했다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	무토큰 목록, 상세 조회	200 공개 조회	200 / 해당 없음(2 건)	PASS	로그 T01
T02	Alice 로그인	200, 토큰과 사용자	200, 토큰과 user 반환, 화면 로그인 표시	PASS	로그 T02, 캡처 02
T03	틀린 비밀번호와 없는 이메일	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS(2 건)	PASS	로그 T03
T04	무토큰 /me, 글 등록	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED(2 건)	PASS	로그 T04

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T05	Alice 글 작성 후 /me	201, authorId 1, /me Alice	201(Location), authorId 1, /me 200	PASS	로그 T05
T06	Alice 본인 글 수정과 삭제	200 과 204, DB 반영	200, 204, 재조회 404	PASS	로그 T06
T07	Bob 이 Alice 글 수정	403, 원문 유지	403 / FORBIDDEN, 원문 유지	PASS	로그 T07
T08	Bob 이 Alice 글 삭제	403, 글 유지	403 / FORBIDDEN, 글 유지	PASS	로그 T08
T09	변조 토큰 4 종 요청	401 INVALID_TOKEN	401 / INVALID_TOKEN(4 종), 변경 없음	PASS	로그 T09
T10	만료 토큰 요청(TTL 3 초)	401 TOKEN_EXPIRED, 화면 해제	401 / TOKEN_EXPIRED(4 종), 화면 해제와 재로그인 안내	PASS	로그 T10, 캡처 07
T11	로그아웃 후 /me	401 TOKEN_REQUIRED	화면 즉시 해제, 401 / TOKEN_REQUIRED	PASS	로그 T11, 캡처 06
T12	등록에 authorId 등 추가	400, 위조 글 미생성	400 / BAD_INPUT(2 건), 위조 글 없음	PASS	로그 T12
T13	빈 제목, 공백, 길이 초과	400 BAD_INPUT	400 / BAD_INPUT(5 건)	PASS	로그 T13
T14	없는 ID 와 잘못된 ID	404 와 400	999999 는 404(3 건), abc 는 400	PASS	로그 T14
T15	서버 재시작 후 조회	글과 작성자 유지	재시작 후 200, 글과 작성자 동일	PASS	로그 T15
T16	로그인 후 새로고침	글 유지, 로그인 해제	글 유지, 화면 로그아웃 전환	PASS	캡처 04
T17	무토큰 변경, Bob 본인 작업	무토큰 401, 본인 작업 성공	무토큰 401(2 건), Bob 201, 200, 204	PASS	로그 T17

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회와 로그인, 수정	200	200(T01~T06, T17)	로그 해당 행
새 글 등록	201 및 Location	201, Location 확인	로그 T05
본인 글 삭제	204 및 빈 본문	204, 빈 본문 확인	로그 T06, T17
입력 오류, 추가 작성자 필드	400 BAD_INPUT	400 / BAD_INPUT	로그 T12, T13, T14
잘못된 계정	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	로그 T03

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
토큰 없음, 헤더 형식 오류	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	로그 T04, T11, T14, T17
변조, 클레임 오류	401 INVALID_TOKEN	401 / INVALID_TOKEN	로그 T09
정상 서명의 만료 토큰	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED	로그 T10
다른 작성자의 변경	403 FORBIDDEN	403 / FORBIDDEN	로그 T07, T08
없는 글	404 NOT_FOUND	404 / NOT_FOUND	로그 T06, T14

보호 API 401 응답(T04, T10)에서 WWW-Authenticate: Bearer 헤더를 확인했다. 실패 응답(T07~T10, T13) 직후 공개 조회로 원문 불변을 확인했다(로그 data unchanged 행). 토큰 없이 없는 글 수정은 401, 정상 토큰으로는 404, Bob 이 Alice 글에 공백 제목을 보내면 403 이었다(로그 T14, EXTRA).

401 과 403 의 화면 차이: 401 이면 request 가 세션을 지워 화면이 로그아웃되고 재로그인 안내가 표시된다(07_token_expired.png). 403 은 로그인 유지 상태로 본인 글만 수정, 삭제할 수 있다는 안내만 표시되며, Bob 에게는 Alice 글의 수정, 삭제 버튼이 보이지 않는다(05_bob_view_alice_post.png).

자동 검사: 수업 제공 도구 py tools/check_api.py --mode final 로 40 개, TOKEN_TTL=3 재시작 후 --expiry 포함 48 개 항목이 모두 통과했다. 화면과 서버 재시작 검사는 별도로 수행했다.

5 AI 활용과 참고 자료

AI 사용 여부: 사용

전체 사용 범위: 과제 수행 중 코드 구현 방법과 예시 코드를 참고하는 데 AI 를 활용하였으며, 작성한 코드의 동작 여부를 확인하였다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT (GPT-6)	코드 작성 시 참고 및 구현 방법 확인	backend/src/auth.js, frontend/src/api.js, frontend/src/App.vue	AI 가 제시한 코드를 참고하여 작성하고, 동작 여부를 확인함

대표 사례 1: 백엔드 인증 구현

도구 / 날짜: ChatGPT (GPT-6) / 2026-10-25

요청한 내용: JWT 발급 및 검증, 사용자 권한 검사 기능의 구현 방법과 예시 코드를 요청하였다.

AI 가 제안한 내용: jsonwebtoken 을 활용한 HS256 방식의 JWT 발급 및 검증, 토큰 만료와 변조에 대한 예외 처리, 사용자 ID 를 이용한 권한 검사 방법을 참고하였다.

실제 반영 위치: backend/src/auth.js

내가 확인하고 수정한 내용: AI 가 제시한 구현 방법과 예시 코드를 참고하여 작성하고, JWT 서명 알고리즘, 토큰 만료 시간, 사용자 ID 검증 및 오류 응답 처리 방식을 확인하였다.

검증 방법과 결과: 자동 검사 48 개 항목과 T01~T17 테스트 결과를 통해 인증 및 권한 검사 기능이 정상적으로 동작하는지 확인하였다.

대표 사례 2: 프런트 로그인 연결

도구 / 날짜: ChatGPT (GPT-6) / 2026-10-25

요청한 내용: 로그인 API 연동, JWT 저장 및 전송, 오류 응답 처리, 게시글 작성자에 따른 수정·삭제 버튼 표시 방법을 질문하였다.

AI 가 제안한 내용: 메모리 기반 토큰 저장, Authorization Bearer 헤더 설정, 401 응답 시 로그인 상태 초기화 및 작성자별 버튼 표시 방법을 참고하였다.

실제 반영 위치: frontend/src/api.js, frontend/src/App.vue

내가 확인하고 수정한 내용: AI 가 제시한 예시 코드를 참고하여 로그인 및 토큰 관리 기능을 작성하고, 로그인, 로그아웃, 새로고침 및 다른 사용자의 게시글에 대한 접근 제한이 정상적으로 처리되는지 확인하였다.

검증 방법과 결과: npm run build 성공 여부와 API 테스트 결과를 확인하고, 로그인 및 토큰 만료와 관련된 테스트(T04, T10, T11)를 통해 동작을 검증하였다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
수업 제공 시작 템플릿과 공통 API 명세(docs/02_API 명세.md)	프로젝트 전체	제공 코드 및 API 요구사항 참고
node-jsonwebtoken, https://github.com/auth0/node-jsonwebtoken	backend/src/auth.js	sign 과 verify 옵션 사용법 참고
Vite 공식 문서, https://vite.dev/guide/	frontend 빌드와 프록시	빌드와 preview 명령 참고

6 이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

   본문의 authorId 를 그대로 저장하면 다른 사람 이름으로 글을 만드는 작성자 위조가 가능해진다. 내 코드는 fields 가 title 과 content 외의 필드를 400 BAD_INPUT 으로 거부하고(T12), 작성자는 본문이 아니라 currentUserId 가 토큰에서 검증해 얻은 ID 로 서버가 결정하므로 위조가 막힌다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

   버튼은 화면 표시일 뿐이라 API 클라이언트로 직접 요청하면 서버에 그대로 도착하므로 서버가 요청마다 검사해야 한다. backend/src/auth.js 의 assertOwner 가 검증된 사용자 ID 와 글의 authorId 를 비교해 다르면 403 FORBIDDEN 을 던지고 DB 를 바꾸지 않으며, T07 과 T08 로 확인했다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

   decode 는 내용을 읽기만 하고 서명을 확인하지 않아 위조 토큰도 통과하지만, verify 는 비밀 키로 서명을 다시 계산해 위조와 만료, 클레임을 검사한다. 내 코드는 jwt.verify 에 HS256 을 고정해 서명, exp, iss, aud 를 검증하고 exp 존재와 sub 형식, 계정 존재를 추가로 확인한다. 변조는 T09(INVALID_TOKEN), 만료는 T10(TOKEN_EXPIRED)으로 확인했다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

   로그아웃은 이 탭의 메모리 토큰만 지우고 서버에는 변화가 없다. 만료는 exp 가 지난 토큰이 모든 요청에서 401 TOKEN_EXPIRED 로 거부되는 것이다. 새로고침은 메모리 초기화로 로그인만 해제되고 SQLite 의 글은 유지된다. 이미 복사된 토큰은 로그아웃해도 exp 까지 유효하므로, 짧은 TTL(90 초)이 유출 피해를 줄인다.

5. 가장 어려웠던 오류 한 가지와 원인 확인, 수정, 재검증 과정을 설명하라.

   백엔드 인증 구현 직후 npm start 가 "JWT_SECRET 은 64 자리 16 진수 문자열이어야 합니다" 오류로 종료되었다. 원인: auth.js 의 secretKey 시작 검사였고, .env 에 .env.example 의 예시 키(16 진수가 아님)가 그대로 남아 있었다. 수정: backend/README.md 의 node 명령으로 새 64 자리 16 진수 키를 만들어 교체했다(키는 저장소에 넣지 않음). 재검증: 서버 정상 시작, 로그인(T02) 200, 자동 검사 48 개 통과를 다시 확인했다.

현재 남은 문제와 다음에 개선할 점: 없음.

제출 확인: [v] 실제 코드와 설명 일치 [v] 테스트 직접 수행 [v] AI 활용 및 출처 기록 [v] 비밀 키와 토큰 제거

GitHub 확인: [v] 루트 README 작성 [v] 보고서 PDF 와 증거 업로드 [v] README 문서 링크 확인 [v] 교수자 접근 가능 [v] 저장소 URL 과 커밋 SHA 제출